Project Brief

Build a small **Go microservice** that exposes a REST API for managing:

- Students
- Courses
- Projects

with the following relationships:

1. One student can take many courses. One course can be taken by many students.
2. One student can have at most one project. One project can be assigned to at most one student.
3. There can be **unassigned** projects.

Use **PostgreSQL** as the database and **GORM** as the ORM.

The service should be production-oriented: clean structure, proper error handling, database migrations, sensible indexes, and basic tests.

You have 2 days.

Functional Requirements

Entities

You must model at least these fields:

- **Student**
  - `id` (UUID or bigserial PK)
  - `student_id` (string or int, unique, e.g. roll number)
  - `name`
  - Timestamps (`created_at`, `updated_at`)
- **Course**
  - `id` (PK)
  - `course_id` (unique)
  - `name`
  - Timestamps
- **Project**
  - `id` (PK)
  - `project_id` (unique)
  - `name`
  - `student_id` (nullable FK, since projects can be unassigned)
  - Timestamps
- **StudentCourses** (join table for many-to-many)
  - `id` (PK)
  - `student_id` (FK → students.id)
  - `course_id` (FK → courses.id)
  - Unique constraint on `(student_id, course_id)`.

> **Key point**: You must explicitly define **primary keys**, **unique keys**, **foreign keys**, and **indexes** where appropriate.

Relationships

- Many-to-many:
  - Students ↔ Courses via `student_courses` table.
- One-to-zero/one:
  - Student ↔ Project via `projects.student_id` (nullable, unique).

Implementation detail:

- Enforce **at most one project per student** by adding a **unique index** on `projects.student_id` (where not null).
- Enforce **at most one student per project** by making `student_id` a normal FK, but each project row represents exactly one project, so one project can only reference one student.

Required Endpoints

You can design the exact routes, but the API must support:

1. CRUD (basic) for each entity

At minimum:

- `POST /students`
- `GET /students/{id}`
- `POST /courses`
- `GET /courses/{id}`
- `POST /projects`
- `GET /projects/{id}`

You don't need full CRUD UI, just JSON APIs.

Example `POST /students` payload:

```
{
  "student_id": "S123",
  "name": "Alice"
}
```

Return created object with internal `id`.

2. Assign courses to a student

An endpoint to enroll a student into courses, e.g.:

- `POST /students/{id}/courses`

Payload:

```
{
  "course_ids": ["C101", "C102"]
}
```

This should:

- Validate courses exist.
- Insert rows into `student_courses`.
- Not create duplicates if called twice with the same course.

3. Assign a project to a student

Endpoint:

- `POST /students/{id}/project`

Payload:

```
{
  "project_id": "P001"
}
```

Rules:

- Fail if project doesn't exist.
- Fail if project is already assigned to another student.
- Fail if student already has a project (unless you explicitly decide to support re-assignment with some logic).

4. Given a student's name or student_id, show their courses and project

Endpoint example:

- `GET /students/details?student_id=S123`
- or `GET /students/details?name=Alice`

Response shape (example):

```
{
  "student": {
    "student_id": "S123",
    "name": "Alice"
  },
  "courses": [
    { "course_id": "C101", "name": "Math" },
    { "course_id": "C102", "name": "Physics" }
  ],
  "project": {
    "project_id": "P001",
    "name": "ML Project"
  }
}
```

If no project: `"project": null`.

Use efficient DB queries (joins / preloads), not N+1 loops.

5. Show all *unassigned* projects

Endpoint:

- `GET /projects/unassigned`

Returns array of projects where `student_id` is `NULL`.

6. Show all courses with *no students*

Endpoint:

- `GET /courses/empty`

Returns all courses that have **no rows in `student_courses`**.

You should write this as a proper SQL query (e.g., `LEFT JOIN` with `WHERE student_courses.course_id IS NULL`) or the equivalent in GORM.

Non-Functional / "Production-ish" Requirements

The goal is to see that you think like a production engineer, not just toy-code.

1. Tech stack and structure

- Language: Go (recent stable version).
- ORM: GORM.
- DB: PostgreSQL.
- Config via **environment variables** (no hard-coded DSNs), e.g.:
  - `DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`.
- Basic folder structure:

```
 /cmd/service/main.go
/internal/config
/internal/db
/internal/models
/internal/handlers
/internal/repositories
/internal/services
```

Even a simpler version is okay, but **keep handlers, DB models, and business logic separated**.

2. Database & migrations

- Use a migration tool or GORM `AutoMigrate`, but:
  - You must show awareness of **constraints** (PK, FK, unique, not null).
- Ensure:

- `students.student_id` is **unique**.
- `courses.course_id` is **unique**.
- `projects.project_id` is **unique**.
- `projects.student_id` has a **unique index** (optional "where not null" if you know how).
- `student_courses(student_id, course_id)` has a **unique constraint**.

3. Indexing

Explicitly define indexes:

- On `students.student_id` (for lookups by student_id).
- On `courses.course_id`.
- On `projects.student_id` (to quickly find a student's project and unassigned projects).
- Rationale: be ready to explain **why** each index exists.

You can add them via:

- GORM tags on models, or
- Migration SQL files.

4. Error handling & HTTP responses

- Use proper HTTP codes:
  - `201` for created.
  - `400` for validation errors.
  - `404` when student/course/project not found.
  - `409` for conflicts (e.g., assigning a project already assigned).
  - `500` for unexpected server errors.
- Responses must be JSON with a clear error structure, e.g.:

```
{
  "error": "student_not_found",
  "message": "Student with id S123 not found"
}
```

5. Logging

- Log at least:
  - Startup and DB connection success.
  - Errors (with enough context).
- Don't print raw SQL errors directly to clients; just log the details.

6. Tests

Minimum tests (you can use an in-memory DB or a test Postgres):

- At least one **unit test** around the service layer:
  - e.g., assigning a project to a student, ensuring constraints are respected.
- At least one **integration test** that hits the DB:
  - e.g., creating a student, course, enrolling, then fetching `/students/details`.

Deliverables

The student should provide:

1. **Source code** in a git repo.
2. `README.md` including:
   - How to run the service (locally and/or via `docker-compose`).
   - Example curl requests or `httpie`/Postman examples.
   - Schema/ERD screenshot or description of relationships.
3. SQL migration files or GORM `AutoMigrate` code with comments.
4. Short note on:
   - Which indexes they added and why.
   - Any design decisions (e.g., why they chose certain types or structure).

Optional but nice:

- `docker-compose.yml` with Postgres + service.
- Makefile with common commands (`run` , `test` , `migrate`).